

Listing 15-1: The Rock, Paper, Scissors game

In this listing, I have snipped the code that creates images, buttons, and text fields for the Splash, Play, and Results scenes. The downloadable files for the book contain the full source code and all the associated art.

Before the program goes into the main loop, we define all three states ❶, instantiate and load all the screen elements, and set the starting state ❷.

We do different event checks depending on which state the program is in. In the Splash state, we only check for clicking the Start button ❸. In the Play state, we check for a click on the Rock, Paper, or Scissors icon buttons ❹. In the Results state, we only check for a click on the Restart button ❺.

Pressing a button or making a selection in one scene changes the value of the `state` variable and therefore moves the game into a different scene. At the bottom of the main loop ❻ ❼ ❽, we draw different screen elements depending on which state the program is currently in.

This technique works well for a small number of states/scenes. However, in a program with more complicated rules or one that has many scenes and/or states, keeping track of what should be done where can become very difficult. Instead, we can take advantage of many of the object-oriented programming techniques introduced earlier in this book and build a different architecture based on independent scenes, all controlled by an object manager object.

A Scene Manager for Managing Many Scenes

The second approach to building a program with multiple scenes is to use a *scene manager*: an object that centralizes the handling of different scenes. We'll create a `SceneMgr` class and instantiate a single `oSceneMgr` object from it. In the following discussion, I'll refer to the `oSceneMgr` object as the scene manager, since we only instantiate one. As you'll see, the scene manager and the related scenes take advantage of encapsulation, inheritance, and polymorphism.

Using the scene manager can be a little tricky, but the resulting program architecture results in a highly modular, easy-to-modify program. A